Q . Simulate decimation by a factor of 2 with anti-alias filtering and analyze the resulting signal

spectrum. (25) (BL3) (WK4)

Given Data:

- **Sampling Frequency = 8 kHz**
- **Signal Frequencies = 500 Hz and 2 kHz**
- **Decimation Factor = 2**

Tasks:

1. Design anti-alias filter.

2. Filter the signal.

3. Perform decimation.

4. Plot spectra before and after decimation.

5. Verify alias prevention.

6. Discuss observations.

Aim

To simulate decimation by a factor of 2 using an anti-aliasing filter and analyze the signal spectrum before and after decimation.

Introduction

Decimation is the process of reducing the sampling rate of a digital signal by an integer factor. Before downsampling, an anti-aliasing low-pass filter is applied to remove high-frequency components that could cause aliasing. In this experiment, a signal sampled at 8 kHz is filtered and then decimated by a factor of 2. The spectra before and after decimation are compared to verify that aliasing is successfully prevented.

Code

```
clc;

clear;
```

```
close all;

%% Given Data
Fs = 8000;          % Original Sampling Frequency (Hz)
F1 = 500;           % Signal Frequency 1 (Hz)
F2 = 2000;          % Signal Frequency 2 (Hz)
D = 2;              % Decimation Factor
Fs_new = Fs/D;      % New Sampling Frequency (Hz)

%% Time Vector
t = 0:1/Fs:0.02;    % 20 ms duration

%% Input Signal
x = sin(2*pi*F1*t) + sin(2*pi*F2*t);

%% 1. Design Anti-Alias Filter
% Cutoff frequency must be less than new Nyquist frequency
Fc = 1800;                      % Cutoff Frequency (Hz)
N = 60;                         % Filter Order
Wn = Fc/(Fs/2);                 % Normalized Cutoff
b = fir1(N,Wn,hamming(N+1));        % FIR Low-pass Filter

%% 2. Filter the Signal
x_filtered = filter(b,1,x);

%% 3. Perform Decimation
x_dec = downsample(x_filtered,D);
```

```
%% Time Vector After Decimation
t_dec = (0:length(x_dec)-1)/Fs_new;

%% 4. Spectrum Before and After Decimation
Nfft = 4096;

X = fftshift(abs(fft(x,Nfft)));
Xf = fftshift(abs(fft(x_filtered,Nfft)));
Xd = fftshift(abs(fft(x_dec,Nfft)));

f = linspace(-Fs/2,Fs/2,Nfft);
f_dec = linspace(-Fs_new/2,Fs_new/2,Nfft);

%% Plot Time Domain Signals
figure;

subplot(3,1,1)
plot(t,x,'b')
title('Original Signal')
xlabel('Time (s)')
ylabel('Amplitude')
grid on

subplot(3,1,2)
plot(t,x_filtered,'r')
title('Filtered Signal')
```

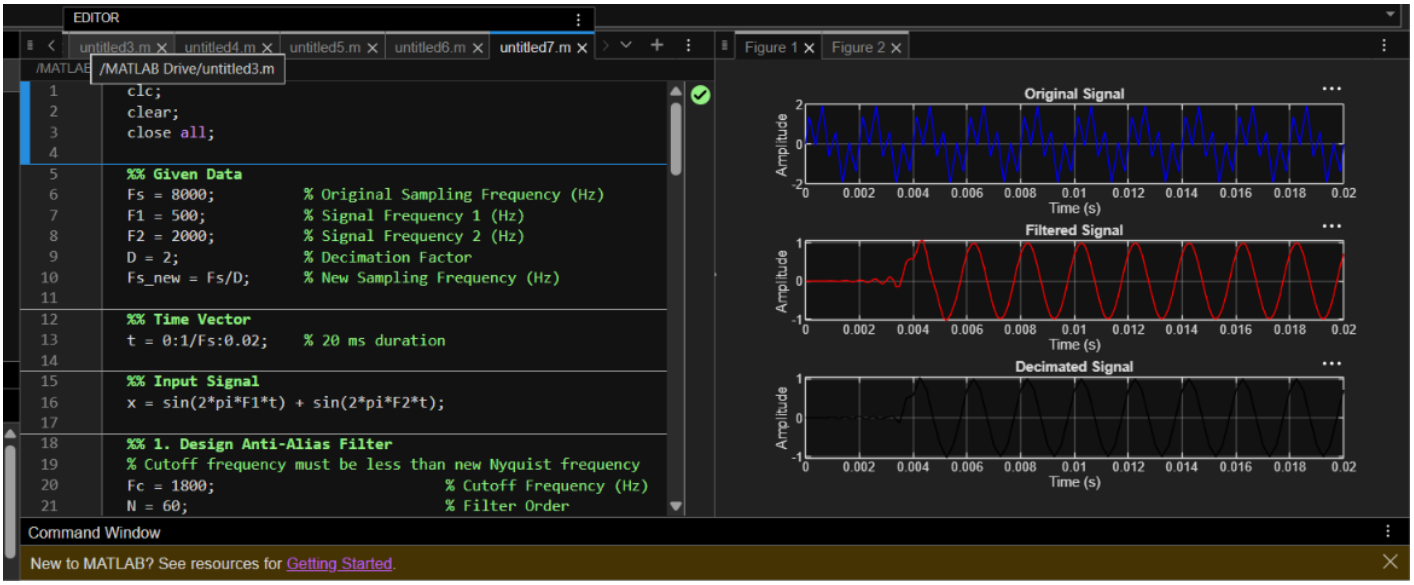
```
xlabel('Time (s)')
ylabel('Amplitude')
grid on

subplot(3,1,3)
plot(t_dec,x_dec,'k')
title('Decimated Signal')
xlabel('Time (s)')
ylabel('Amplitude')
grid on

%% Plot Frequency Spectra
figure;

subplot(3,1,1)
plot(f,X/max(X),'b')
title('Spectrum Before Filtering')
xlabel('Frequency (Hz)')
ylabel('Normalized Magnitude')
grid on
xlim([-4000 4000])

subplot(3,1,2)
plot(f,Xf/max(Xf),'r')
title('Spectrum After Anti-Alias Filter')
xlabel('Frequency (Hz)')
ylabel('Normalized Magnitude')
```

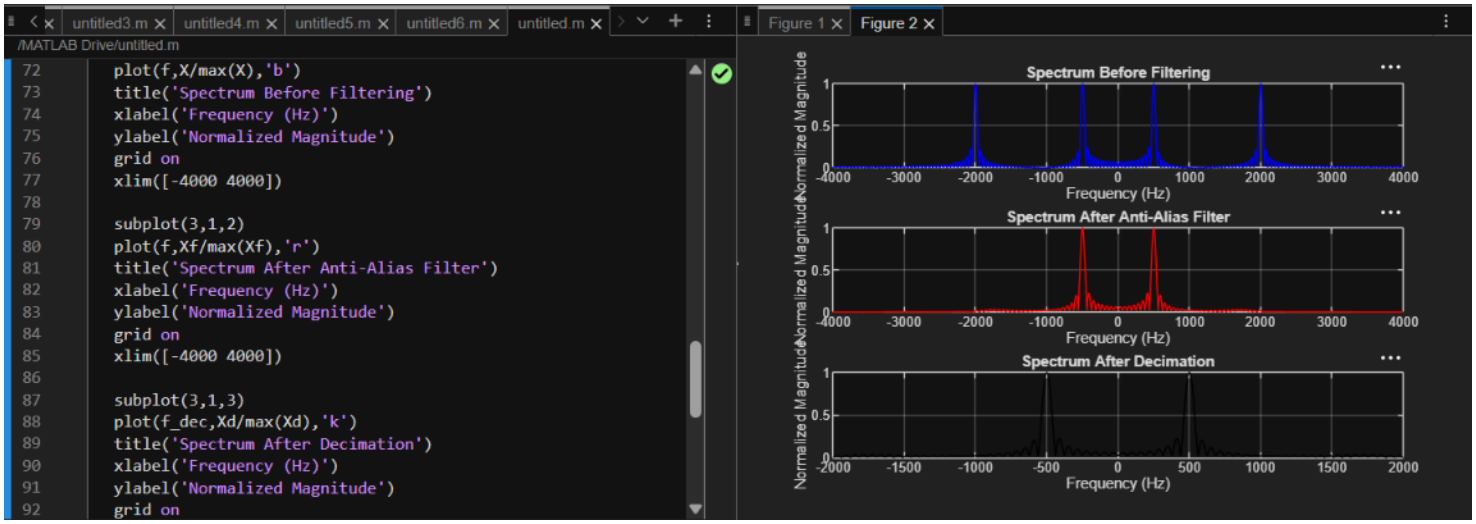
```
grid on
xlim([-4000 4000])

subplot(3,1,3)
plot(f_dec,Xd/max(Xd),'k')
title('Spectrum After Decimation')
xlabel('Frequency (Hz)')
ylabel('Normalized Magnitude')
grid on
xlim([-2000 2000])
```

Result

The decimation by a factor of 2 was successfully simulated using an FIR anti-aliasing low-pass filter. The filter effectively removed high-frequency components before downsampling, preventing aliasing. The frequency spectrum after decimation showed that the desired signal components were preserved without spectral distortion, confirming the effectiveness of the anti-alias filtering process.

output


EDITOR
untitled3.m
untitled4.m
untitled5.m
untitled6.m
untitled7.m
/MATLAB Drive/untitled3.m
clc;
clear;
close all;
%% Given Data
Fs = 8000; % Original Sampling Frequency (Hz)
F1 = 500; % Signal Frequency 1 (Hz)
F2 = 2000; % Signal Frequency 2 (Hz)
D = 2; % Decimation Factor
Fs_new = Fs/D; % New Sampling Frequency (Hz)
%% Time Vector
t = 0:1/Fs:0.02; % 20 ms duration
%% Input Signal
x = sin(2*pi*F1*t) + sin(2*pi*F2*t);
%% 1. Design Anti-Alias Filter
% Cutoff frequency must be less than new Nyquist frequency
Fc = 1800; % Cutoff Frequency (Hz)
N = 60; % Filter Order
Figure 1
Figure 2
Original Signal
Filtered Signal
Decimated Signal
Amplitude
Time (s)
Command Window
New to MATLAB? See resources for Getting Started.


untitled3.m
untitled4.m
untitled5.m
untitled6.m
untitled.m
/MATLAB Drive/untitled.m
plot(f,X/max(X),'b')
title('Spectrum Before Filtering')
xlabel('Frequency (Hz)')
ylabel('Normalized Magnitude')
grid on
xlim([-4000 4000])
subplot(3,1,2)
plot(f,Xf/max(Xf),'r')
title('Spectrum After Anti-Alias Filter')
xlabel('Frequency (Hz)')
ylabel('Normalized Magnitude')
grid on
xlim([-4000 4000])
subplot(3,1,3)
plot(f_dec,Xd/max(Xd),'k')
title('Spectrum After Decimation')
xlabel('Frequency (Hz)')
ylabel('Normalized Magnitude')
grid on
Figure 1
Figure 2
Spectrum Before Filtering
Spectrum After Anti-Alias Filter
Spectrum After Decimation
Normalized Magnitude
Frequency (Hz)